Comprehensive Notes on Vectors

For Machine Learning, Deep Learning & AI

Master-Level Reference Guide

Topics Covered

Vector Operations • Norms & Distances • Dot Products
Projections • Vector Spaces • ML/DL Applications

Compiled by: Akshay Redekar
December 14, 2025

Contents

1 Introduction to Vectors **3**
1.1 Why Vectors Matter in ML/DL/AI . . . 3
1.2 Notation and Conventions . . . 3

2 Vector Representations **3**
2.1 Column Vectors vs Row Vectors . . . 3
2.2 Special Vectors . . . 4

3 Vector Magnitude and Distance **4**
3.1 Vector Norms (Magnitudes) . . . 4
3.2 Distance Between Vectors . . . 6

4 Vector Operations **6**
4.1 Scalar-Vector Operations . . . 6
4.2 Vector-Vector Operations . . . 7
4.3 Dot Product (Inner Product) . . . 7

5 Angles and Projections **9**
5.1 Angle Between Vectors . . . 9
5.2 Unit Vectors and Normalization . . . 10
5.3 Vector Projection . . . 10

6 Linear Independence and Basis **11**
6.1 Linear Combinations . . . 11
6.2 Linear Independence . . . 11
6.3 Span . . . 12
6.4 Basis . . . 12

7 Vector Spaces **12**
7.1 Subspaces . . . 12
7.2 Hyperplanes and Decision Boundaries . . . 13

8 Python Implementation **13**
8.1 NumPy Basics . . . 13
8.2 Distance and Similarity Functions . . . 14
8.3 Machine Learning Example: Linear Regression . . . 14
8.4 Deep Learning: Neural Network Forward Pass . . . 15
8.5 Word Embeddings Example . . . 16
8.6 K-NN Classification . . . 16

9 Advanced ML/DL Applications **17**
9.1 Embedding Spaces . . . 17
9.2 Attention Mechanisms . . . 17
9.3 Gradient Descent Optimization . . . 18

10 Assessment Questions **18**
10.1 Basic Level (1-10) 18
10.2 Intermediate Level (11-20) 19
10.3 Advanced Level (21-30) 19
10.4 Solutions to Selected Questions 20

11 Key Takeaways **20**

12 References and Further Reading **21**
12.1 Books 21
12.2 Papers 22
12.3 Online Resources 22

A Mathematical Proofs **22**
A.1 Cauchy-Schwarz Inequality 22

B Additional Code Examples **22**
B.1 PyTorch Implementation 22

1 Introduction to Vectors

Definition 1.1 (Vector)**.** A **vector** is a mathematical object characterized by both **magnitude** (size) and **direction**. In machine learning and data science, vectors are ordered lists of numbers used to represent data points, features, embeddings, and model parameters.

1.1 Why Vectors Matter in ML/DL/AI

Machine Learning Applications

- **Data Representation**: Each data sample is a feature vector
- **Distance Calculations**: KNN, K-Means clustering
- **Linear Models**: Weights and bias terms as vectors
- **Dimensionality Reduction**: PCA, t-SNE operate on vectors

Deep Learning Applications

- **Neural Network Layers**: Input/output activations as vectors
- **Embeddings**: Word2Vec, BERT produce dense vectors
- **Gradients**: Backpropagation computes gradient vectors
- **Attention Mechanisms**: Query, Key, Value vectors

1.2 Notation and Conventions

- **Scalar**: Single number, lowercase italic: x, y, α
- **Vector**: Ordered array, bold lowercase: $\mathbf{v}, \mathbf{x}, \mathbf{w}$
- **Matrix**: 2D array, bold uppercase: $\mathbf{A}, \mathbf{W}$
- **Components**: Subscript notation: $v_1, v_2, \ldots, v_n$
- **Transpose**: $\mathbf{v}^\top$ converts column $\leftrightarrow$ row

2 Vector Representations

2.1 Column Vectors vs Row Vectors

Vector Forms

Column Vector (default in ML/DL): $n \times 1$ matrix

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n$$

Row Vector: $1 \times n$ matrix

$$\mathbf{v}^\top = \begin{bmatrix} v_1 & v_2 & v_3 & \cdots & v_n \end{bmatrix}$$

Critical Shape Rule

In matrix multiplication, shapes must align:

$$(m \times n) \times (n \times p) = (m \times p)$$

Most DL errors stem from shape mismatches!

Example 2.1 (Housing Data Feature Vector)**.**

Representing a house with 4 features:

$$\mathbf{x}_{\text{house}} = \begin{bmatrix} \text{Area (sqft)} \\ \text{Bedrooms} \\ \text{Bathrooms} \\ \text{Age (years)} \end{bmatrix} = \begin{bmatrix} 1500 \\ 3 \\ 2 \\ 10 \end{bmatrix}$$

Dimension: $\mathbf{x} \in \mathbb{R}^4$ (4-dimensional feature vector)

2.2 Special Vectors

Type	Definition	ML Example
Zero Vector	$\mathbf{0} = [0, 0, \ldots, 0]^\top$	Initial bias
Unit Vector	$\|\hat{\mathbf{v}}\| = 1$	Normalized embeddings
Standard Basis	$\mathbf{e}_i$ has 1 at position i, else 0	One-hot encoding
Sparse Vector	Most components = 0	Bag-of-words in NLP
Dense Vector	Few zeros	Word2Vec, BERT embeddings

Table 1: Common vector types in ML/DL

3 Vector Magnitude and Distance

3.1 Vector Norms (Magnitudes)

Definition 3.1 (Norm)**.** A **norm** is a function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ satisfying:

1. $\|\mathbf{v}\| \geq 0$ (non-negativity), $\|\mathbf{v}\| = 0 \iff \mathbf{v} = \mathbf{0}$

2. $\|c\mathbf{v}\| = |c| \, \|\mathbf{v}\|$ (absolute homogeneity)
3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangle inequality)

Standard Norms

L1 Norm (Manhattan Distance):

$$\|\mathbf{v}\|_1 = \sum_{i=1}^{n} |v_i|$$

L2 Norm (Euclidean Distance):

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{n} v_i^2} = \sqrt{\mathbf{v}^\top \mathbf{v}}$$

Lp Norm (General):

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^{n} |v_i|^p \right)^{1/p}$$

L∞ Norm (Maximum):

$$\|\mathbf{v}\|_\infty = \max_i |v_i|$$

ML/DL Applications of Norms

- **L1 Regularization (Lasso)**: Promotes sparsity, feature selection

$$\mathcal{L}_{\text{Lasso}} = \text{MSE} + \lambda \, \|\mathbf{w}\|_1$$

- **L2 Regularization (Ridge)**: Prevents large weights, improves generalization

$$\mathcal{L}_{\text{Ridge}} = \text{MSE} + \lambda \, \|\mathbf{w}\|_2^2$$

- **Gradient Clipping**: Prevents exploding gradients in RNNs

$$\text{if } \|\nabla\|_2 > \text{threshold}: \quad \nabla \leftarrow \frac{\text{threshold}}{\|\nabla\|_2} \nabla$$

- **Batch Normalization**: Uses variance (related to L2 norm)

Example 3.1 (Computing Norms)**.**

Given $\mathbf{v} = [3, -4, 2]$:

L1 Norm:

$$\|\mathbf{v}\|_1 = |3| + |-4| + |2| = 3 + 4 + 2 = 9$$

L2 Norm:

$$\|\mathbf{v}\|_2 = \sqrt{3^2 + (-4)^2 + 2^2} = \sqrt{9 + 16 + 4} = \sqrt{29} \approx 5.385$$

L∞ Norm:

$$\|\mathbf{v}\|_\infty = \max(|3|, |-4|, |2|) = 4$$

3.2 Distance Between Vectors

Euclidean Distance

The distance between vectors $\mathbf{u}$ and $\mathbf{v}$:

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{n} (u_i - v_i)^2}$$

Manhattan Distance

$$d_1(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_1 = \sum_{i=1}^{n} |u_i - v_i|$$

Distance Metrics in ML

- **K-Nearest Neighbors (KNN)**: Classification based on closest training points
- **K-Means Clustering**: Assigns points to nearest cluster centroid
- **Mean Squared Error (MSE)**: MSE $= \frac{1}{m} \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$
- **Anomaly Detection**: Points far from cluster centers

Example 3.2 (Distance Calculation)**.**

Given $\mathbf{u} = [1, 2]$ and $\mathbf{v} = [4, 6]$:
Euclidean Distance:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{(4-1)^2 + (6-2)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Manhattan Distance:

$$d_1(\mathbf{u}, \mathbf{v}) = |4 - 1| + |6 - 2| = 3 + 4 = 7$$

4 Vector Operations

4.1 Scalar-Vector Operations

Addition/Subtraction (Shifting)

$$\mathbf{v} + c = \begin{bmatrix} v_1 + c \\ v_2 + c \\ \vdots \\ v_n + c \end{bmatrix}, \quad \mathbf{v} - c = \begin{bmatrix} v_1 - c \\ v_2 - c \\ \vdots \\ v_n - c \end{bmatrix}$$

Effect: Shifts all components by scalar c

Multiplication/Division (Scaling)

$$k\mathbf{v} = \begin{bmatrix} kv_1 \\ kv_2 \\ \vdots \\ kv_n \end{bmatrix}, \quad \frac{\mathbf{v}}{k} = \begin{bmatrix} v_1/k \\ v_2/k \\ \vdots \\ v_n/k \end{bmatrix}$$

Effect: Scales magnitude by $|k|$; reverses direction if $k < 0$

ML Applications

- **Feature Scaling**: Normalize features to same range
- **Mean Centering**: $\mathbf{x}_{\text{centered}} = \mathbf{x} - \mu$
- **Learning Rate**: $\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha\nabla\mathcal{L}$
- **Bias Addition**: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$

4.2 Vector-Vector Operations

Element-wise Addition/Subtraction

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_1 + v_1 \\ u_2 + v_2 \\ \vdots \\ u_n + v_n \end{bmatrix}, \quad \mathbf{u} - \mathbf{v} = \begin{bmatrix} u_1 - v_1 \\ u_2 - v_2 \\ \vdots \\ u_n - v_n \end{bmatrix}$$

Properties:

- Commutative: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
- Associative: $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
- Identity: $\mathbf{v} + \mathbf{0} = \mathbf{v}$

Deep Learning Applications

- **ResNet Skip Connections**: $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$
- **Error Vectors**: $\mathbf{e} = \mathbf{y}_{\text{true}} - \mathbf{y}_{\text{pred}}$
- **Gradient Updates**: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla$

4.3 Dot Product (Inner Product)

Definition 4.1 (Dot Product)**.** The **dot product** of vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ is:

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i$$

Output: Scalar value measuring similarity/alignment

Geometric Interpretation

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta$$

where θ is the angle between vectors

Key Properties:

- $\mathbf{u} \cdot \mathbf{v} > 0$: Acute angle (similar direction)
- $\mathbf{u} \cdot \mathbf{v} = 0$: Orthogonal (perpendicular)
- $\mathbf{u} \cdot \mathbf{v} < 0$: Obtuse angle (opposite direction)

Most Important ML/DL Operation!

Neural Network Forward Pass:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$$

Attention Mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Cosine Similarity:

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

[Dot Product Properties] For vectors $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^n$ and scalar c:

1. **Commutative**: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$
2. **Distributive**: $\mathbf{u} \cdot (\mathbf{v} + \mathbf{w}) = \mathbf{u} \cdot \mathbf{v} + \mathbf{u} \cdot \mathbf{w}$
3. **Scalar Multiplication**: $(c\mathbf{u}) \cdot \mathbf{v} = c(\mathbf{u} \cdot \mathbf{v})$
4. **Positive Definite**: $\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|_2^2 \geq 0$

Example 4.1 (Dot Product Calculation)**.**

Problem: Compute $\mathbf{a} \cdot \mathbf{b}$ for $\mathbf{a} = [1, 2, 3]$ and $\mathbf{b} = [4, 0, 1]$

Solution:

$$\mathbf{a} \cdot \mathbf{b} = (1)(4) + (2)(0) + (3)(1) = 4 + 0 + 3 = 7$$

Interpretation: Positive dot product indicates vectors point in similar direction.

Example 4.2 (Neural Network Neuron)**.**

Setup:

- Input: $\mathbf{x} = [0.5, 0.3, 0.2]^\top$
- Weights: $\mathbf{w} = [0.4, 0.6, -0.3]^\top$
- Bias: $b = 0.1$

Pre-activation:

$$z = \mathbf{w}^\top \mathbf{x} + b = (0.4)(0.5) + (0.6)(0.3) + (-0.3)(0.2) + 0.1$$

$$z = 0.2 + 0.18 - 0.06 + 0.1 = 0.42$$

Activation (ReLU):

$$a = \max(0, z) = \max(0, 0.42) = 0.42$$

5 Angles and Projections

5.1 Angle Between Vectors

Angle Formula

The angle θ between non-zero vectors $\mathbf{u}$ and $\mathbf{v}$:

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2}$$

$$\theta = \arccos\left(\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2}\right)$$

Cosine Similarity

Measures directional similarity, ignoring magnitude:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2} \in [-1, 1]$$

Applications:

- Recommendation systems (user-item similarity)
- Document similarity in NLP
- Image retrieval systems
- Word embedding comparisons

Example 5.1 (Angle Calculation)**.**

Given: $\mathbf{a} = [1, 1]$ and $\mathbf{b} = [1, 0]$
Step 1 - Dot Product:

$$\mathbf{a} \cdot \mathbf{b} = (1)(1) + (1)(0) = 1$$

Step 2 - Magnitudes:

$$\|\mathbf{a}\|_2 = \sqrt{1^2 + 1^2} = \sqrt{2}, \quad \|\mathbf{b}\|_2 = \sqrt{1^2} = 1$$

Step 3 - Cosine:

$$\cos\theta = \frac{1}{\sqrt{2} \cdot 1} = \frac{1}{\sqrt{2}} \approx 0.707$$

Step 4 - Angle:

$$\theta = \arccos(0.707) = 45 = \frac{\pi}{4} \text{ radians}$$

5.2 Unit Vectors and Normalization

Definition 5.1 (Unit Vector)**.** A vector with magnitude 1 is called a **unit vector**. Any non-zero vector $\mathbf{v}$ can be normalized:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

where $\|\hat{\mathbf{v}}\|_2 = 1$

Normalization in Deep Learning

Why Normalize?

- **Stable Training**: Prevents exploding/vanishing activations
- **Direction Focus**: Removes magnitude effects
- **Cosine Similarity**: Requires unit vectors
- **Batch Normalization**: Normalizes layer inputs

L2 Normalization Layer:

$$\mathbf{y} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2 + \epsilon}$$

where ϵ prevents division by zero

Example 5.2 (Vector Normalization)**.**

Given: $\mathbf{v} = [3, 4]$

Step 1 - Magnitude:

$$\|\mathbf{v}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

Step 2 - Normalize:

$$\hat{\mathbf{v}} = \frac{1}{5}\begin{bmatrix}3\\4\end{bmatrix} = \begin{bmatrix}0.6\\0.8\end{bmatrix}$$

Verification:

$$\|\hat{\mathbf{v}}\|_2 = \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = \sqrt{1} = 1 \quad \checkmark$$

5.3 Vector Projection

Definition 5.2 (Scalar Projection)**.** The **scalar projection** of $\mathbf{b}$ onto $\mathbf{a}$ is:

$$\text{comp}_{\mathbf{a}}\mathbf{b} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2}$$

Interpretation: Length of $\mathbf{b}$'s shadow on $\mathbf{a}$

Definition 5.3 (Vector Projection)**.** The **vector projection** of $\mathbf{b}$ onto $\mathbf{a}$ is:

$$\text{proj}_{\mathbf{a}}\mathbf{b} = \left(\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2^2}\right)\mathbf{a} = \left(\frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}}\right)\mathbf{a}$$

Interpretation: Component of $\mathbf{b}$ in direction of $\mathbf{a}$

Principal Component Analysis (PCA)

PCA finds directions of maximum variance by projecting data onto principal components (eigenvectors of covariance matrix):
Algorithm:

1. Center data: $\mathbf{X}_{\text{centered}} = \mathbf{X} - \mu$
2. Compute covariance: $\mathbf{C} = \frac{1}{m}\mathbf{X}^\top\mathbf{X}$
3. Find eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$
4. Project: $\mathbf{z} = \mathbf{V}^\top\mathbf{x}$ where $\mathbf{V} = [\mathbf{v}_1 \cdots \mathbf{v}_k]$

Example 5.3 (Vector Projection)**.**

Given: $\mathbf{b} = [0, 3]$ and $\mathbf{a} = [4, 0]$
Scalar Projection:

$$\text{comp}_{\mathbf{a}}\mathbf{b} = \frac{(4)(0) + (0)(3)}{\sqrt{16}} = \frac{0}{4} = 0$$

Vector Projection:

$$\text{proj}_{\mathbf{a}}\mathbf{b} = \left(\frac{0}{16}\right)\begin{bmatrix}4\\0\end{bmatrix} = \begin{bmatrix}0\\0\end{bmatrix}$$

Interpretation: $\mathbf{b}$ is perpendicular to $\mathbf{a}$, so projection is zero vector.

6 Linear Independence and Basis

6.1 Linear Combinations

Definition 6.1 (Linear Combination)**.** A **linear combination** of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

where $c_1, \ldots, c_k \in \mathbb{R}$ are scalars (coefficients).

6.2 Linear Independence

Definition 6.2 (Linear Independence)**.** Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are **linearly independent** if:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k = \mathbf{0} \quad \Rightarrow \quad c_1 = c_2 = \cdots = c_k = 0$$

Otherwise, they are **linearly dependent**.

Multicollinearity in Linear Regression

Problem: Linearly dependent features cause:

- Unstable coefficient estimates
- Non-invertible design matrix $\mathbf{X}^\top\mathbf{X}$
- High variance in predictions

Solution: Remove redundant features or use regularization

6.3 Span

Definition 6.3 (Span)**.** The **span** of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is the set of all linear combinations:

$$\text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\} = \left\{ \sum_{i=1}^{k} c_i \mathbf{v}_i : c_i \in \mathbb{R} \right\}$$

Interpretation: All vectors reachable by combining $\mathbf{v}_1, \ldots, \mathbf{v}_k$

6.4 Basis

Definition 6.4 (Basis)**.** A **basis** for vector space V is a linearly independent set that spans V.

[Standard Basis] In $\mathbb{R}^n$, the standard basis is:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \ldots, \quad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

Any vector $\mathbf{v} = \sum_{i=1}^{n} v_i \mathbf{e}_i$

Feature Spaces

- **Original Feature Space**: $\mathbf{x} \in \mathbb{R}^d$ with standard basis
- **PCA Basis**: New orthogonal basis aligned with variance
- **Kernel Methods**: Implicit high-dimensional feature space

7 Vector Spaces

Definition 7.1 (Vector Space)**.** A **vector space** V over field $\mathbb{F}$ (usually $\mathbb{R}$) is a set with:

- **Vector Addition**: $\mathbf{u}, \mathbf{v} \in V \Rightarrow \mathbf{u} + \mathbf{v} \in V$
- **Scalar Multiplication**: $c \in \mathbb{F}, \mathbf{v} \in V \Rightarrow c\mathbf{v} \in V$

satisfying 8 axioms (associativity, commutativity, distributivity, etc.)

7.1 Subspaces

Definition 7.2 (Subspace)**.** $W \subseteq V$ is a **subspace** if:

1. $\mathbf{0} \in W$
2. Closed under addition: $\mathbf{u}, \mathbf{v} \in W \Rightarrow \mathbf{u} + \mathbf{v} \in W$
3. Closed under scalar multiplication: $c \in \mathbb{R}, \mathbf{v} \in W \Rightarrow c\mathbf{v} \in W$

In $\mathbb{R}^3$:

Example 7.1 (Common Subspaces).

- Lines through origin: 1-dimensional subspaces
- Planes through origin: 2-dimensional subspaces
- $\{\mathbf{0}\}$ and $\mathbb{R}^3$ itself: trivial subspaces

7.2 Hyperplanes and Decision Boundaries

Definition 7.3 (Hyperplane). A **hyperplane** in $\mathbb{R}^n$ is:

$$\{\mathbf{x} \in \mathbb{R}^n : \mathbf{w}^\top \mathbf{x} + b = 0\}$$

where $\mathbf{w} \in \mathbb{R}^n$ (normal vector) and $b \in \mathbb{R}$ (bias).

Support Vector Machines (SVM)

SVM finds the optimal separating hyperplane:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

Decision Function:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

Objective: Maximize margin $\frac{2}{\|\mathbf{w}\|_2}$

8 Python Implementation

8.1 NumPy Basics

```python
import numpy as np

# Create vectors
v = np.array([1, 2, 3])
w = np.array([4, 5, 6])

# Vector operations
v_plus_w = v + w            # [5 7 9]
v_minus_w = v - w           # [-3 -3 -3]
scalar_mult = 2 * v         # [2 4 6]

# Dot product
dot_product = np.dot(v, w)  # 32
# Alternative: v @ w

# Norms
l1_norm = np.linalg.norm(v, 1)        # 6.0
l2_norm = np.linalg.norm(v)           # 3.7416573867739413
linf_norm = np.linalg.norm(v, np.inf) # 3.0

```

```python
# Unit vector
unit_v = v / np.linalg.norm(v)
print(f"Unit vector: {unit_v}")
# [0.26726124 0.53452248 0.80178373]
```

8.2 Distance and Similarity Functions

```python
def euclidean_distance(u, v):
    """Compute L2 distance between vectors"""
    return np.linalg.norm(u - v)

def manhattan_distance(u, v):
    """Compute L1 distance between vectors"""
    return np.sum(np.abs(u - v))

def cosine_similarity(u, v):
    """Compute cosine similarity"""
    return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

def cosine_distance(u, v):
    """Compute cosine distance"""
    return 1 - cosine_similarity(u, v)

# Example usage
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

print(f"Euclidean: {euclidean_distance(u, v):.4f}")
print(f"Manhattan: {manhattan_distance(u, v):.4f}")
print(f"Cosine Sim: {cosine_similarity(u, v):.4f}")
```

8.3 Machine Learning Example: Linear Regression

```python
# Linear Regression: y = w^T x + b
def predict(X, w, b):
    """
    X: feature matrix (m x n)
    w: weight vector (n x 1)
    b: bias scalar
    Returns: predictions (m x 1)
    """
    return np.dot(X, w) + b

# Example dataset
X = np.array([
    [1500, 3, 2],  # House 1: area, bedrooms, bathrooms
    [2000, 4, 3],  # House 2
    [1200, 2, 1]   # House 3
])
```

```
17
18 # Weights (learned from training)
19 w = np.array([50, 30000, 20000])  # price per: sqft, bedroom,
       bathroom
20 b = 10000  # base price
21
22 # Predictions
23 prices = predict(X, w, b)
24 print("Predicted prices:", prices)
25 # [135000. 190000. 100000.]
```

8.4 Deep Learning: Neural Network Forward Pass

```
1  def sigmoid(z):
2      """Sigmoid activation function"""
3      return 1 / (1 + np.exp(-z))
4
5  def relu(z):
6      """ReLU activation function"""
7      return np.maximum(0, z)
8
9  def forward_pass(x, W, b, activation='relu'):
10     """
11     Single layer forward pass
12     x: input vector (n,)
13     W: weight matrix (m x n)
14     b: bias vector (m,)
15     Returns: activations (m,)
16     """
17     z = np.dot(W, x) + b  # Pre-activation
18
19     if activation == 'relu':
20         return relu(z)
21     elif activation == 'sigmoid':
22         return sigmoid(z)
23     else:
24         return z  # Linear
25
26 # Example
27 x = np.array([0.5, 0.3, 0.2])
28 W = np.array([
29     [0.4, 0.6, -0.3],
30     [0.1, -0.2, 0.5]
31 ])
32 b = np.array([0.1, -0.1])
33
34 output = forward_pass(x, W, b, activation='relu')
35 print(f"Layer output: {output}")
36 # [0.42 0.  ]
```

8.5 Word Embeddings Example

```python
from sklearn.metrics.pairwise import cosine_similarity

# Simplified word vectors (in practice: 300D from Word2Vec)
word_vectors = {
    'king': np.array([0.5, 0.1, 0.8]),
    'queen': np.array([0.4, 0.2, 0.7]),
    'man': np.array([0.4, 0.0, 0.6]),
    'woman': np.array([0.3, 0.1, 0.5])
}

# Word analogy: king - man + woman     ?
result = (word_vectors['king'] - word_vectors['man']
          + word_vectors['woman'])

# Find closest word
max_sim = -1
closest_word = None

for word, vec in word_vectors.items():
    if word == 'king':
        continue
    sim = cosine_similarity(result.reshape(1, -1),
                            vec.reshape(1, -1))[0][0]
    if sim > max_sim:
        max_sim = sim
        closest_word = word

print(f"king - man + woman     {closest_word}")
print(f"Similarity: {max_sim:.4f}")
# Expected: queen
```

8.6 K-NN Classification

```python
from collections import Counter

def knn_classify(X_train, y_train, x_query, k=3):
    """
    K-Nearest Neighbors classification
    X_train: training features (m x n)
    y_train: training labels (m,)
    x_query: query point (n,)
    k: number of neighbors
    """
    # Compute distances to all training points
    distances = [np.linalg.norm(x_query - x) for x in X_train]

    # Get indices of k nearest neighbors
    k_indices = np.argsort(distances)[:k]
```

```
16
17      # Get labels of k nearest neighbors
18      k_labels = [y_train[i] for i in k_indices]
19
20      # Majority vote
21      most_common = Counter(k_labels).most_common(1)
22      return most_common[0][0]
23
24  # Example
25  X_train = np.array([
26      [1, 2], [3, 3], [5, 1], [2, 4]
27  ])
28  y_train = np.array([0, 1, 0, 1])
29  x_query = np.array([2, 3])
30
31  prediction = knn_classify(X_train, y_train, x_query, k=3)
32  print(f"Predicted class: {prediction}")
```

9 Advanced ML/DL Applications

9.1 Embedding Spaces

Word2Vec Architecture

Skip-Gram Model: Predict context words from center word

Objective:

$$\max \frac{1}{T} \sum_{t=1}^{T} \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t)$$

Probability (softmax):

$$P(w_o | w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i=1}^{V} \exp(\mathbf{u}_i^\top \mathbf{v}_c)}$$

where $\mathbf{v}_c$ is center word vector, $\mathbf{u}_o$ is context word vector

9.2 Attention Mechanisms

Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Components:

- Query vectors: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$
- Key vectors: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$
- Value vectors: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$

Intuition: Dot product $\mathbf{Q}\mathbf{K}^\top$ measures relevance between query and keys

9.3 Gradient Descent Optimization

Gradient Descent Update

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t)$$

where:

- $\mathbf{w}_t$: weight vector at iteration t
- α: learning rate (scalar)
- $\nabla_{\mathbf{w}} \mathcal{L}$: gradient vector of loss

Momentum

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta)\nabla_{\mathbf{w}} \mathcal{L}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_{t+1}$$

Accumulates past gradients to smooth updates

10 Assessment Questions

10.1 Basic Level (1-10)

1. What is the L2 norm of $\mathbf{v} = [1, 2, 2]$?
2. Given $\mathbf{a} = [2, 5]$ and $\mathbf{b} = [3, 1]$, compute:
   (a) $\mathbf{a} + \mathbf{b}$
   (b) $2\mathbf{a} - \mathbf{b}$
   (c) $\mathbf{a} \cdot \mathbf{b}$
3. Convert $\mathbf{v} = [3, 4]$ to a unit vector.
4. True/False: The dot product of two vectors is always positive.
5. What is the geometric interpretation of the dot product?

6. Calculate the Euclidean distance between $\mathbf{u} = [0, 0]$ and $\mathbf{v} = [3, 4]$.
7. What does it mean for two vectors to be orthogonal?
8. Compute: $3 \times [1, 2, 3]$
9. What is the L1 norm of $\mathbf{v} = [1, -2, 3]$?
10. State the triangle inequality for vector norms.

10.2 Intermediate Level (11-20)

11. Calculate the angle between $\mathbf{a} = [1, 0]$ and $\mathbf{b} = [1, 1]$.
12. Given vectors $\mathbf{v} = [1, 2, 3]$ and $\mathbf{w} = [2, -1, 1]$, compute:
    (a) Dot product
    (b) Angle between them
    (c) Euclidean distance
13. Explain the difference between L1 and L2 regularization in terms of vector norms.
14. In a neural network, if $\mathbf{x} = [1, 2]$, $\mathbf{w} = [0.5, 0.3]$, and $b = 0.1$, what is $z = \mathbf{w}^\top \mathbf{x} + b$?
15. Calculate cosine similarity between:
    - doc1 $= [1, 2, 0, 3]$
    - doc2 $= [2, 1, 1, 2]$
16. What is the projection of $\mathbf{b} = [0, 3]$ onto $\mathbf{a} = [4, 0]$?
17. Check if vectors $[1, 1]$ and $[2, 2]$ are linearly independent.
18. Normalize the vector $\mathbf{v} = [1, 2, 2]$ to unit length.
19. What is the range of cosine similarity values?
20. In gradient descent, what does the learning rate α represent geometrically?

10.3 Advanced Level (21-30)

21. Derive the gradient of $\mathcal{L} = \left\|\mathbf{y} - \mathbf{w}^\top \mathbf{x}\right\|_2^2$ with respect to $\mathbf{w}$.
22. Prove that $\|k\mathbf{v}\|_2 = |k| \, \|\mathbf{v}\|_2$.
23. Explain how PCA uses vector projections for dimensionality reduction.
24. In a 2-layer neural network:
    - Input: $\mathbf{x} = [1, 2, 3]$
    - $\mathbf{W}_1 = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{bmatrix}$
    - $\mathbf{b}_1 = [0.1, 0.2]^\top$

- Activation: sigmoid

Calculate the output of the first layer.

25. Compare one-hot encoding vs word embeddings for NLP. Include dimensionality and semantic considerations.

26. Given word vectors:

- king = $[0.5, 0.1, 0.8]$
- man = $[0.4, 0.0, 0.6]$
- woman = $[0.3, 0.0, 0.5]$

Calculate: king - man + woman. What word should this approximate?

27. Explain why the Gram-Schmidt process is important for creating orthogonal bases.

28. Design a feature vector for a movie recommendation system. What features would you include?

29. Describe how vectors are used in the attention mechanism of Transformers.

30. In k-NN with $k = 3$:

- Query: $[2, 3]$
- Training: $[(1, 2), (3, 3), (5, 1), (2, 4)]$
- Labels: $[0, 1, 0, 1]$

Classify the query point using Euclidean distance.

10.4 Solutions to Selected Questions

Question 1: $\|\mathbf{v}\|_2 = \sqrt{1^2 + 2^2 + 2^2} = \sqrt{9} = 3$
Question 3: $\hat{\mathbf{v}} = \frac{1}{5}[3, 4]^\top = [0.6, 0.8]^\top$
Question 4: False. Can be negative (obtuse angle) or zero (orthogonal).
Question 11: $\cos\theta = \frac{1}{\sqrt{2}} \Rightarrow \theta = 45$
Question 14: $z = 0.5 \times 1 + 0.3 \times 2 + 0.1 = 1.2$
Question 26: Result $\approx [0.4, 0.1, 0.7]$ should be close to "queen"

11 Key Takeaways

Essential Concepts

1. **Vectors** are fundamental to ML/DL/AI data representation
2. **Dot product** is the most important operation:
   - Neural network computations
   - Similarity measures
   - Attention mechanisms

3. **Norms** control model behavior:
    - L1: sparsity (Lasso)
    - L2: stability (Ridge)
    - Used in regularization and gradient clipping
4. **Distance metrics** power algorithms:
    - KNN classification
    - K-Means clustering
    - Loss functions (MSE)
5. **Cosine similarity** measures directional similarity:
    - Recommendation systems
    - Document retrieval
    - Embedding comparisons
6. **Projections** enable dimensionality reduction (PCA)
7. **Linear independence** prevents multicollinearity
8. **Vector spaces** provide mathematical foundation for feature transformations

For Deep Learning Practitioners:

- Understand vector shapes for debugging matrix operations
- Use normalization to stabilize training
- Monitor gradient norms to detect exploding/vanishing gradients
- Leverage dot products for efficient computations
- Think of neural layers as vector space transformations

12 References and Further Reading

12.1 Books

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 2: Linear Algebra.
- Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.

12.2 Papers

- Mikolov, T., et al. (2013). "Efficient Estimation of Word Representations in Vector Space." *arXiv:1301.3781*.
- Pennington, J., Socher, R., & Manning, C. D. (2014). "GloVe: Global Vectors for Word Representation." *EMNLP*.
- Vaswani, A., et al. (2017). "Attention Is All You Need." *NeurIPS*.

12.3 Online Resources

- 3Blue1Brown: Essence of Linear Algebra (YouTube series)
- Stanford CS229: Machine Learning (Lecture notes)
- Fast.ai: Practical Deep Learning for Coders
- MIT OpenCourseWare: Linear Algebra (Gilbert Strang)

A Mathematical Proofs

A.1 Cauchy-Schwarz Inequality

Theorem A.1 (Cauchy-Schwarz)**.** For any vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$:

$$|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\|_2 \|\mathbf{v}\|_2$$

with equality if and only if $\mathbf{u}$ and $\mathbf{v}$ are linearly dependent.

Proof. Consider $f(t) = \|\mathbf{u} + t\mathbf{v}\|_2^2 \geq 0$ for all $t \in \mathbb{R}$.

Expanding:

$$f(t) = \|\mathbf{u}\|_2^2 + 2t(\mathbf{u} \cdot \mathbf{v}) + t^2 \|\mathbf{v}\|_2^2$$

This is a quadratic in t that is always non-negative, so its discriminant must be non-positive:

$$4(\mathbf{u} \cdot \mathbf{v})^2 - 4 \|\mathbf{u}\|_2^2 \|\mathbf{v}\|_2^2 \leq 0$$

Therefore:

$$(\mathbf{u} \cdot \mathbf{v})^2 \leq \|\mathbf{u}\|_2^2 \|\mathbf{v}\|_2^2$$

Taking square roots completes the proof. □

B Additional Code Examples

B.1 PyTorch Implementation

```
import torch

# Create tensors (PyTorch's vectors)
v = torch.tensor([1.0, 2.0, 3.0])
w = torch.tensor([4.0, 5.0, 6.0])
```

```
6
7  # Operations
8  dot_product = torch.dot(v, w)
9  l2_norm = torch.norm(v, p=2)
10 cosine_sim = torch.nn.functional.cosine_similarity(
11     v.unsqueeze(0), w.unsqueeze(0)
12 )
13
14 print(f"Dot product: {dot_product.item()}")
15 print(f"L2 norm: {l2_norm.item()}")
16 print(f"Cosine similarity: {cosine_sim.item()}")
```